From Numerical Baselines to Gradient-Boosted Trees for Heart-Disease Classification

Jacobo Romero Díaz[1] Pablo González Monfort[1] Roger Baiges Trilla[1]
[1] EPFL, School of Computer and Communication Sciences
{jacobo.romerodiaz, pablo.gonzalezmonfort, roger.baigestrilla}@epfl.ch

***Abstract*—We classify self-reported heart disease in the 2015 BRFSS survey using six numerical methods and a NumPy gradient-boosted tree ensemble. The selected GBDT improves validation F1 from 0.4386 for logistic regression to 0.4462. On the test set, logistic regression scores 0.449 F1 and GBDT scores 0.454.**

I. Introduction

Cardiovascular disease is a major public-health concern, and the 2015 Behavioral Risk Factor Surveillance System (BRFSS) provides a large survey for studying associated health factors [1]. Its 321 predictors cover demographics, behaviors such as smoking and physical activity, reported conditions such as diabetes and hypertension, and healthcare use. The course release has 328,135 labeled respondents and 109,379 unlabeled test respondents, while the target variable `_MICHD` indicates a self-reported history of coronary heart disease or heart attack.

This dataset mixes numeric quantities, categorical answer codes, and structured missingness. For example, `PHYSHLTH`=88 means zero days, while 77 and 99 mean an unknown or refused answer. Codebook-based rules drop 14 survey-design variables, two duplicate variables, and 14 raw mixed-unit variables with computed replacements; they also convert units and fill numeric values implied by skipped questions. This leaves 56 numeric and 235 categorical predictors. Unknown and refused categorical answers remain separate categories. Moreover, only 8.83% of training labels are positive showing a clear data unbalance.

Linear models such as logistic regression are common baselines for tabular classification, while tree ensembles can capture nonlinear effects and interactions. Gradient-boosted decision trees (GBDT) are well suited to these survey data because they can split the cleaned variables directly and learn where to route missing answers [2], [3]. We then compare both approaches.

II. Methods

A. Linear baselines.

We implement the six methods required by the project as linear baselines. Let X include an intercept, let $y \in \{0,1\}^N$, and write $z = Xw$. The methods minimize either squared error or logistic loss:

$$L_{\mathrm{MSE}}(w) = \frac{\|Xw - y\|_2^2}{2N}, \tag{1}$$

$$L_{\log}(w) = \frac{1}{N}\sum_{i=1}^{N}\left[\log(1 + e^{z_i}) - y_i z_i\right]. \tag{2}$$

For MSE, gradient descent (GD) uses the full gradient in $w \leftarrow w - \gamma \nabla L_{\mathrm{MSE}}(w)$, while stochastic GD (SGD) uses one sampled row per update. Least squares instead uses a pseudoinverse of $X^\top X$, which is singular for our one-hot design; ridge adds a regularizer $\lambda\|w\|_2^2$ and solves $(X^\top X + 2N\lambda I)w = X^\top y$. The other two methods minimize logistic loss by GD, with or without the same L_2 penalty.

In the experiments, MSE GD uses 100 steps at $\gamma = 0.02$ and SGD uses 10^6 single-row steps at $\gamma = 5 \cdot 10^{-6}$. Both logistic methods use 300 steps at $\gamma = 0.1$; ridge and regularized logistic regression use $\lambda = 10^{-3}$.

B. Gradient-Boosted Decision Trees (GBDT).

GBDT models log-odds as an additive sum of trees, starting from the training prevalence: $F_{t+1}(x) = F_t(x) + \eta f_t(x)$ [4]. Each tree is fitted to reduce logistic loss. Our NumPy code uses the loss gradients and Hessians to score candidate splits and leaf values [2]. It bins each cleaned variable into at most 64 values, so split search uses compact histograms [3]. Missing answers get a separate bin, routed to whichever branch gives greater loss reduction. Depth-5 trees can capture interactions between survey answers. We use 550 trees, $\eta = 0.05$, leaf L_2 penalty 10, minimum child Hessian 5, and row/feature fractions 0.7/0.3; logits from seeds 0–2 are averaged.

C. Data processing

Each fold fits preprocessing on training rows and applies it to validation rows. Linear models fill missing numeric values with training medians, add missingness flags, one-hot encode categories seen at least 30 times, and standardize the resulting $\sim$ 1,530 columns. GBDT instead splits the 291 cleaned variables directly and learns how to route missing values.

D. Validation

Only 8.83% of labels are positive, so predicting all negatives gives 91.17% accuracy but zero positive-class F1. We therefore report F1 and threshold-independent ROC AUC. The numerical baselines use five shuffled folds (seed 42), with each decision threshold fitted on training scores. The GBDT comparison repeats five folds with split seeds 42–44; each scored fold uses a threshold chosen from the other folds' out-of-fold predictions. Test F1 comes from submitted predictions scored against hidden labels.

III. Results

Table I
NUMERICAL BASELINES ON FIVE VALIDATION FOLDS (SEED 42). F1 IS MEAN ± FOLD SD; ROC AUC IS THE FOLD MEAN. ONLY THE RAW LEAST-SQUARES BASELINE USES UNENCODED INPUTS.

Method	F1 (mean ± SD)	AUC
Raw inputs / LS	0.4095 ± 0.0019	0.845
MSE / GD	0.4289 ± 0.0022	0.858
MSE / SGD	0.4255 ± 0.0033	0.855
Least squares	0.4301 ± 0.0018	0.857
Ridge	0.4297 ± 0.0031	0.858
Logistic / GD	**0.4390** ± 0.0027	0.864
L2 logistic / GD	0.4388 ± 0.0022	0.864

Representation and linear baselines.: Encoding raises least-squares F1 from 0.4095 on raw inputs to 0.4301 (Table I); logistic GD reaches 0.4390, the best linear result. The tested L_2 penalty changes logistic F1 by only −0.0002. GD, SGD, and direct least squares fit the same MSE model, so their differences partly reflect finite optimization budgets.

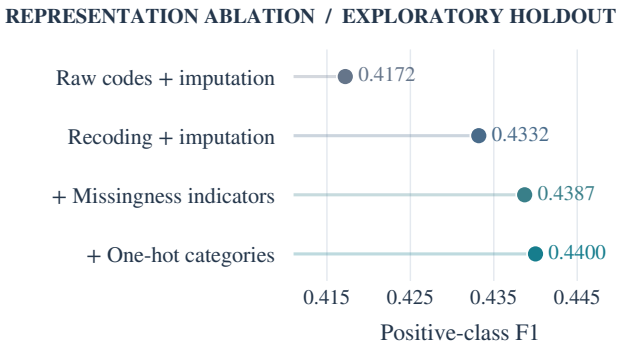


Figure 1. Exploratory logistic-regression F1 on one 80/20 holdout. The threshold was optimized on the holdout, making F1 optimistic. Recoding also removes survey-design fields and changes categorical unknown codes, unlike the final pipeline.

Figure 1 shows an exploratory rise from 0.4172 to 0.4400 F1 after recoding, missingness flags, and one-hot encoding. Because several changes were combined, it does not isolate their effects. A separate five-fold check found only −0.0003 F1 from numeric cleaning once one-hot encoding was present.

Table II
MODEL COMPARISON ON 15 VALIDATION FOLDS (FIVE FOLDS WITH THREE SPLIT SEEDS) AND ON SUBMITTED TEST PREDICTIONS. CV THRESHOLDS COME FROM OTHER FOLDS. BOLD MARKS THE FINAL MODEL; DASHES MEAN NO REPORTED SCORE.

Model	CV F1	CV AUC	Test F1
Logistic regression	0.4386	0.8643	0.449
Single GBDT (9-run mean)	0.4457	0.8698	–
Three-seed GBDT	**0.4462**	**0.8703**	**0.454**
GBDT + logistic blend	0.4463	0.8703	–
Seven-config GBDT bag	0.4469	–	–
GBDT + neural-net blend	0.4469	–	–
Single GBDT + new features	0.4466–0.4468	–	–
GBDT feature bag (3/5 seeds)	0.4474	–	–

Model choice and refinements.: The five-fold baseline table and the 15-fold model comparison use different threshold procedures, so their F1 values should be read within each table. On the 15 shared folds, the selected three-seed GBDT improves F1 over logistic regression by 0.0077 (corrected 95% interval $[0.0038, 0.0115]$) [5]. The test submissions score 0.449 F1 for logistic regression and 0.454 for this GBDT.

The nine single-GBDT runs span 0.4449–0.4469 F1; averaging seeds adds only 0.0001 over one selected seed. The feature bag uses answer counts and target-ordered codes and reaches 0.4474, but its paired gain is only 0.0011 (corrected $t = 1.51$). Other bags and blends in Table II show similarly small gains. A four-config bag gained 0.0020 in one five-fold comparison, with an interval that includes zero; using logistic predictions as a GBDT input gave 0.4456–0.4460, and mixing feature and config bags gave 0.4467. We retain the three-seed model because no alternative shows a reliable CV gain and `run.py` reproduces its test submission. Model selection used these same folds, so CV may overestimate performance.

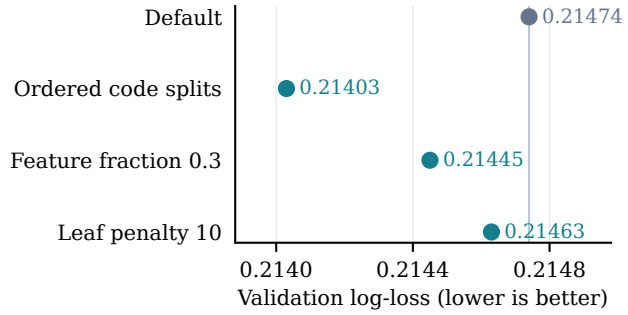


Figure 2. One-factor GBDT tuning on five validation folds (seed 42). Each point is mean log-loss at its best tree count. Changes are evaluated separately, not cumulatively.

Ordered-code splits lower best-stage validation log-loss from 0.21474 to 0.21403 and raise F1 from 0.4456 to 0.4467 (Fig. 2). A feature fraction of 0.3 reaches the same F1; stronger leaf regularization helps less.

A. Further GBDT checks

Other depth, sampling, binning, and child-Hessian changes show no clear gain. Starting GBDT from logistic predictions performs worse (best-stage F1 0.4439).

Reducing the learning rate to 0.02 with about 2,000 trees lowers validation log-loss from 0.21377 to 0.21349, but nested F1 stays near 0.4456. We therefore use rate 0.05 and 550 trees, within a flat 500–900-tree region; extending the default model to 2,000 trees overfits (nested F1 0.4323). Depth-one trees score 0.4417 nested F1 versus 0.4457 for the selected depth-five single model, although other settings differ. In separate ablations, depth two reaches best-stage F1 0.4445, while removing `HAREHAB1` yields 0.4372. Best-stage F1 chooses its threshold on the evaluated fold and is optimistic relative to nested F1.

IV. Conclusions

Fold-fitted encoding gives a useful linear baseline, but GBDT provides the main F1 gain. More complex ensembles and features show no reliable CV improvement. The final `run.py` refits three GBDTs on all labeled rows, averages their logits, and applies threshold 0.2166 to `x_test.csv`; its reported test F1 is 0.454.

This is a retrospective survey task. All 773 training respondents who answered `HAREHAB1`, a question about rehabilitation after a heart attack, are positive. That signal limits interpretation of the result as prospective clinical risk prediction.

REFERENCES

[1] Centers for Disease Control and Prevention, ''Behavioral Risk Factor Surveillance System: 2015 Codebook Report,'' U.S. Department of Health and Human Services, Tech. Rep., 2016. [Online]. Available: https://www.cdc.gov/brfss/annual_data/2015/pdf/codebook15_llcp.pdf

[2] T. Chen and C. Guestrin, ''XGBoost: A scalable tree boosting system,'' in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [Online]. Available: https://arxiv.org/abs/1603.02754

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, ''LightGBM: A highly efficient gradient boosting decision tree,'' in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[4] J. H. Friedman, ''Greedy function approximation: A gradient boosting machine,'' *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[5] C. Nadeau and Y. Bengio, ''Inference for the generalization error,'' *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003. [Online]. Available: https://link.springer.com/article/10.1023/A:1024068626366